

Course Title: AI Assisted Coding

Course Code: 23CS002PC304

Faculty Name: Dr. R. Prashant Kumar

Name: P. Ashwitha

HT no: 2303A52420- Batch(36)

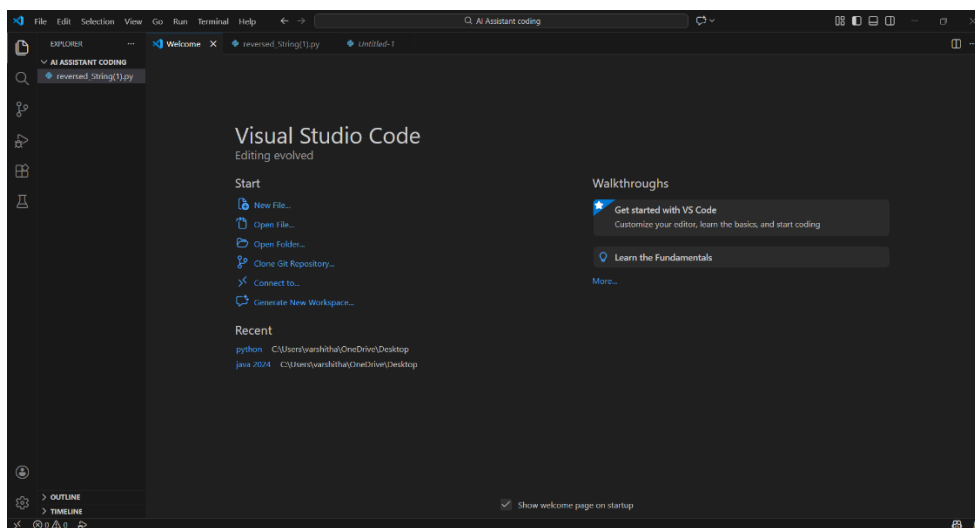
Question:

Lab 1: Environment Setup – GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow

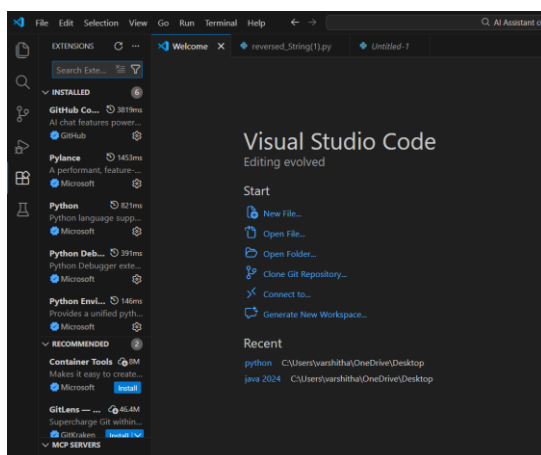
Task 0:

- Install and configure GitHub Copilot in VS Code. Take screenshots of each step.

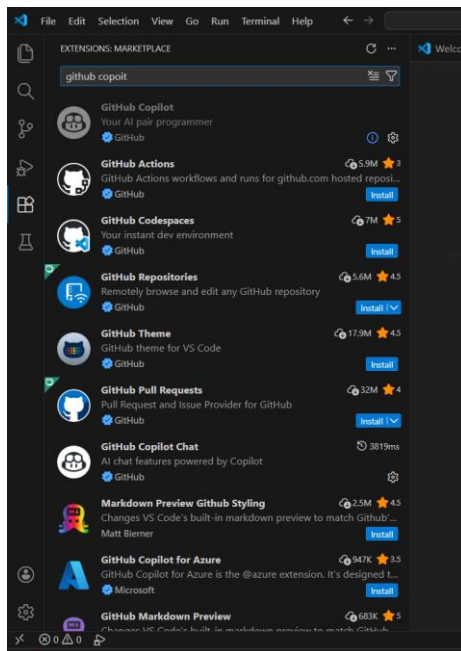
Step 1: Open Visual Studio Code



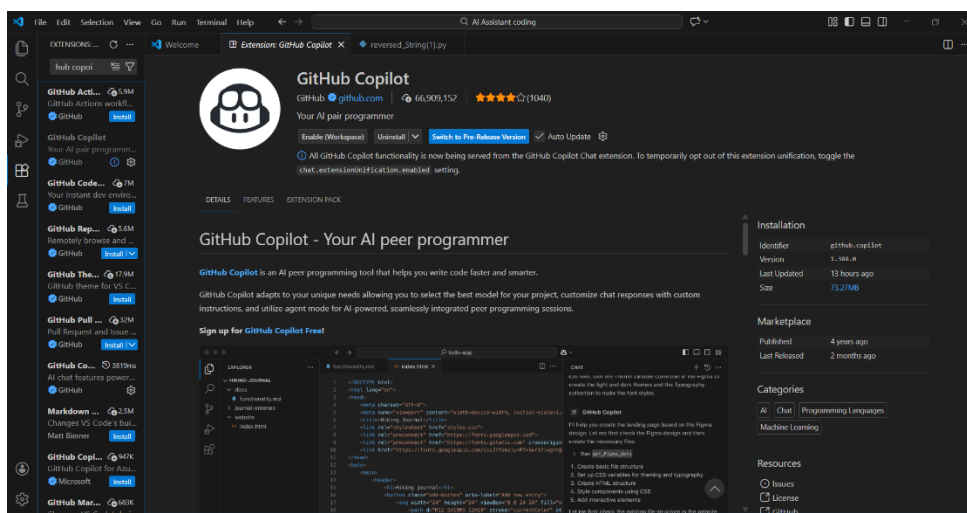
Step 2: Open Extensions Panel



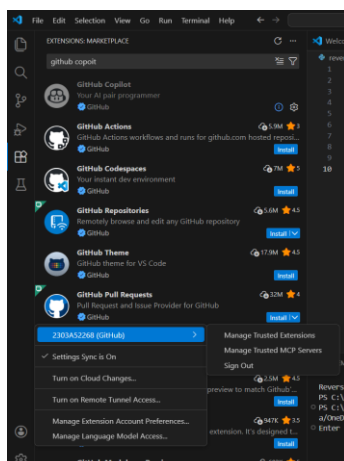
Step 3: Search for GitHub Copilot



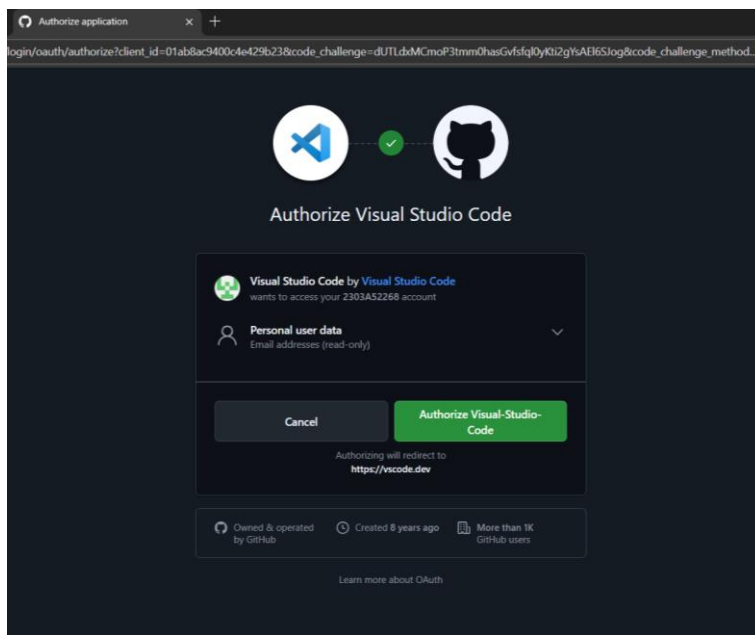
Step 4: Install GitHub Copilot



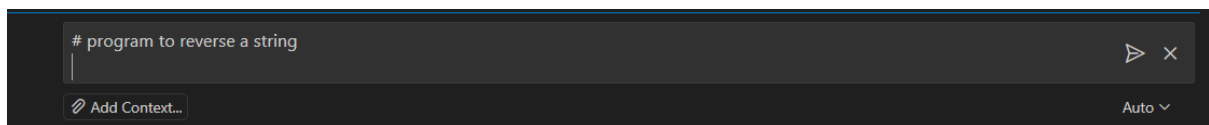
Step 5: Sign in to GitHub Account



Step 6: Authorize GitHub Copilot

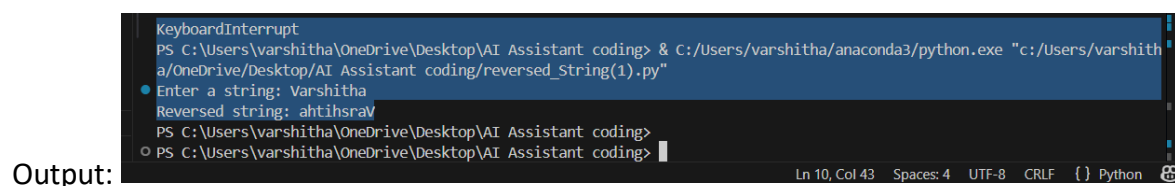
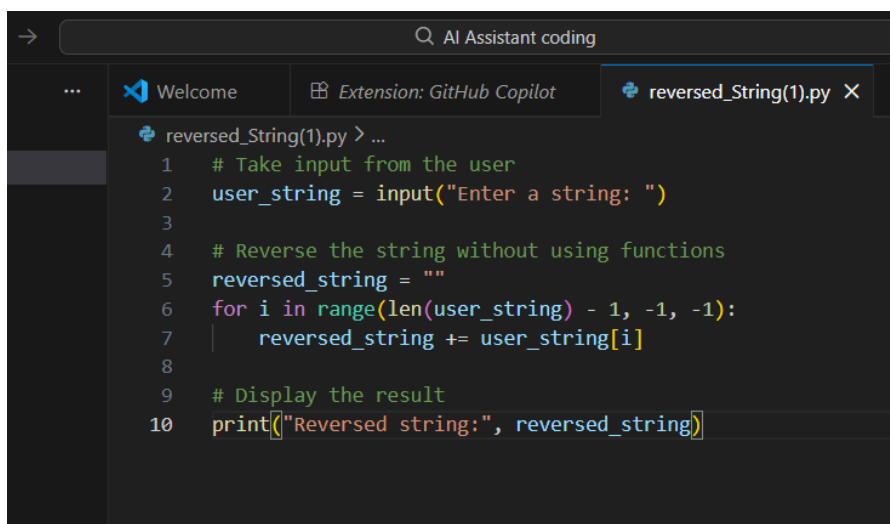


Step 7: Verify Copilot is Enabled



Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

program to reverse a string



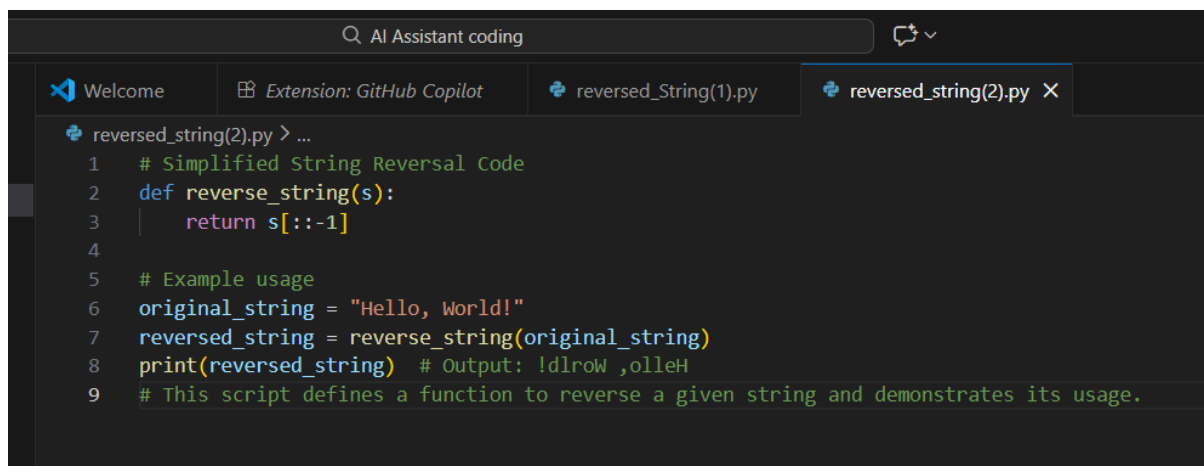
Output:

Explanation

- The input() function takes a string from the user.
- An empty string rev is created to store the reversed result.
- The for loop iterates through the string from the last character to the first.
- Each character is appended to rev.
- The final reversed string is printed.
- The logic is written directly in the main code without using functions

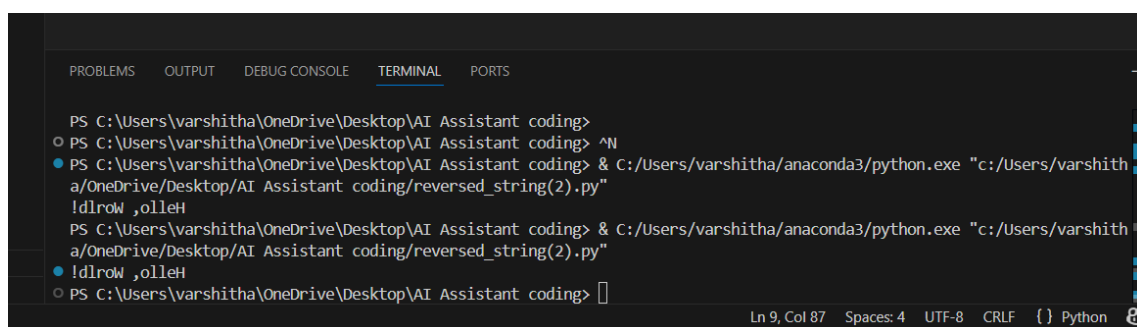
Task 2: Efficiency & Logic Optimization (Readability Improvement)

Simplified String Reversal Code



```
AI Assistant coding
Welcome  Extension: GitHub Copilot  reversed_string(1).py  reversed_string(2).py X
reversed_string(2).py > ...
1  # Simplified String Reversal Code
2  def reverse_string(s):
3      return s[::-1]
4
5  # Example usage
6  original_string = "Hello, World!"
7  reversed_string = reverse_string(original_string)
8  print(reversed_string) # Output: !dlroW ,olleH
9  # This script defines a function to reverse a given string and demonstrates its usage.
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding>
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> ^N
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(2).py"
!dlroW ,olleH
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(2).py"
!dlroW ,olleH
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding>
Ln 9, Col 87  Spaces: 4  UTF-8  CRLF  {} Python
```

Explanation of Optimization

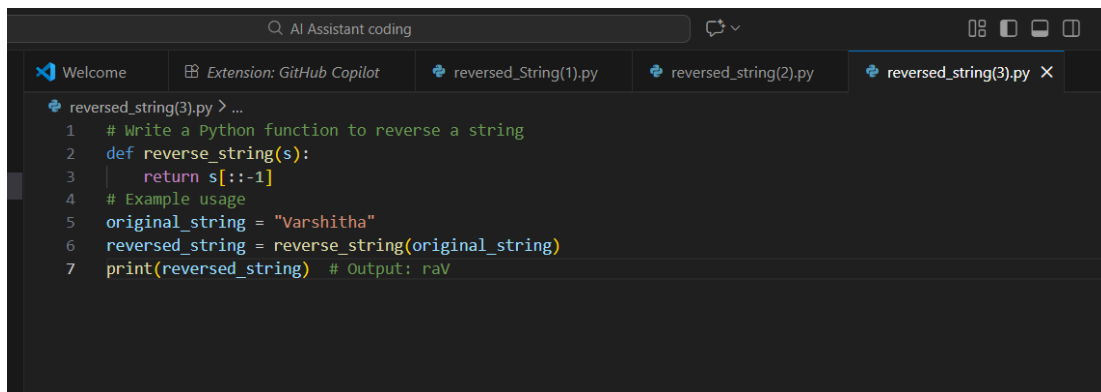
- The loop and extra variable were removed
- Python slicing reverses the string in a single step
- Code is shorter, cleaner, and easier to understand

Time Complexity Explanation

- Original code: **O(n)** (manual loop)
- Optimized code: **O(n)** (built-in slicing)
- Although complexity remains the same, slicing is **faster in practice** due to internal optimization

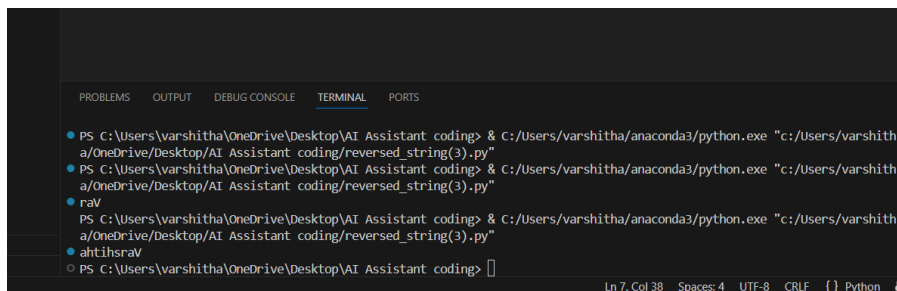
Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Write a Python function to reverse a string



```

1 # Write a Python function to reverse a string
2 def reverse_string(s):
3     return s[::-1]
4 # Example usage
5 original_string = "Varshitha"
6 reversed_string = reverse_string(original_string)
7 print(reversed_string) # Output: raV
  
```



```

PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(3).py"
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(3).py"
raV
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(3).py"
ahtisraV
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding>
  
```

Output:

Explanation

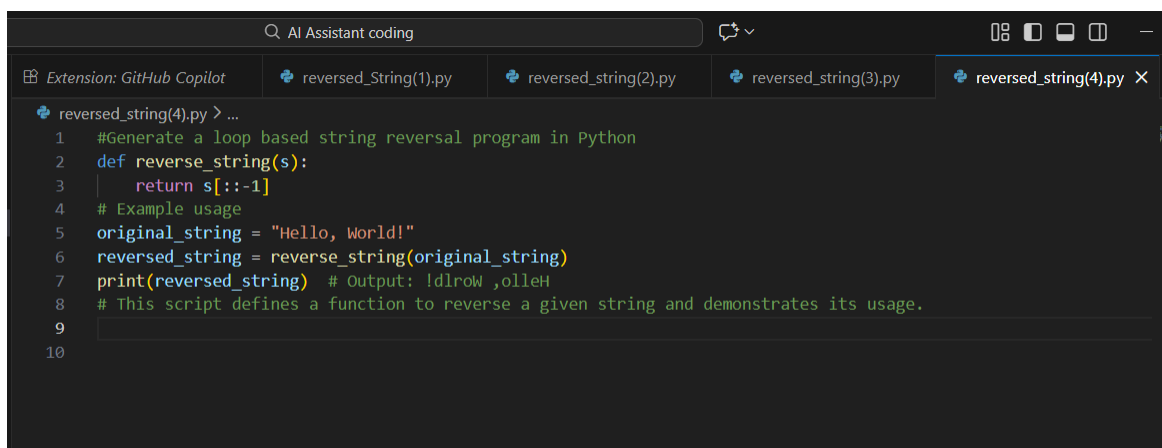
- A function `reverse_string()` is defined to reverse a string.
- The function takes one parameter `text`.
- The slicing method `[::-1]` is used to reverse the string.
- The reversed string is returned to the caller.
- User input is passed to the function.
- The result is printed.
- This modular approach improves reusability and readability.

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

Aspect	Without Function (Procedural)	With Function (Modular)
Code Clarity	Moderate	High
Reusability	Not reusable	Highly reusable
Debugging	Difficult	Easier
Maintainability	Low	High
Large-scale Suitability	Poor	Good

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

#Generate a loop based string reversal program in Python

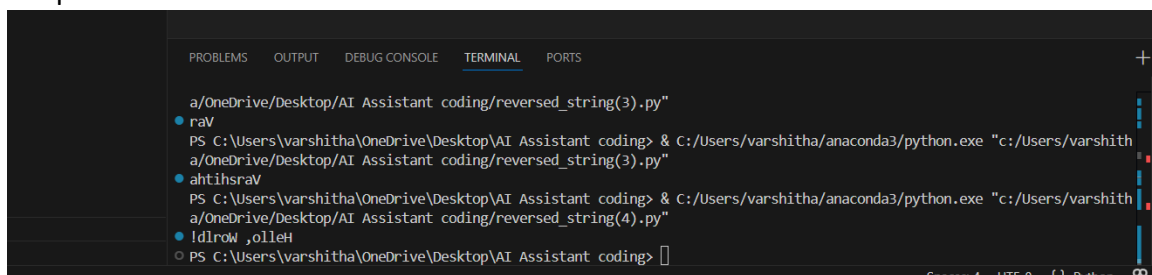


```

1 #Generate a loop based string reversal program in Python
2 def reverse_string(s):
3     return s[::-1]
4 # Example usage
5 original_string = "Hello, World!"
6 reversed_string = reverse_string(original_string)
7 print(reversed_string) # Output: dlrow ,olleH
8 # This script defines a function to reverse a given string and demonstrates its usage.
9
10

```

Output:



```

a/OneDrive/Desktop/AI Assistant coding/reversed_string(3).py
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/reversed_string(3).py"
dlrow ,olleH
PS C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding>

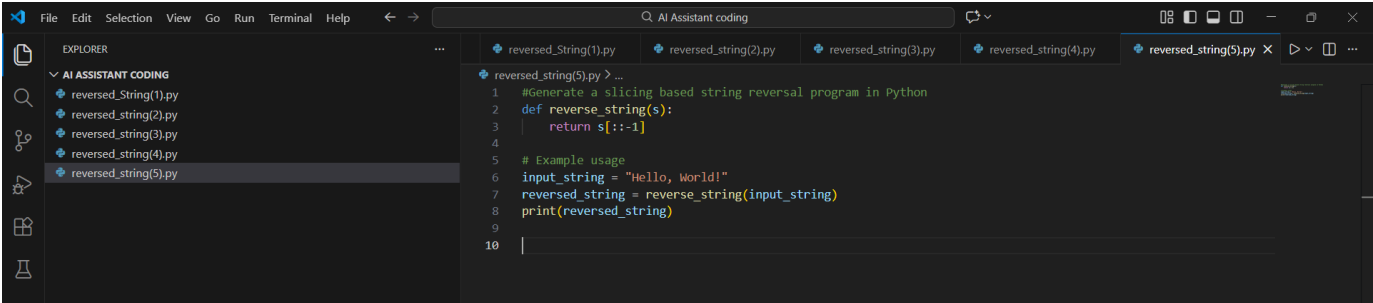
```

Explanation

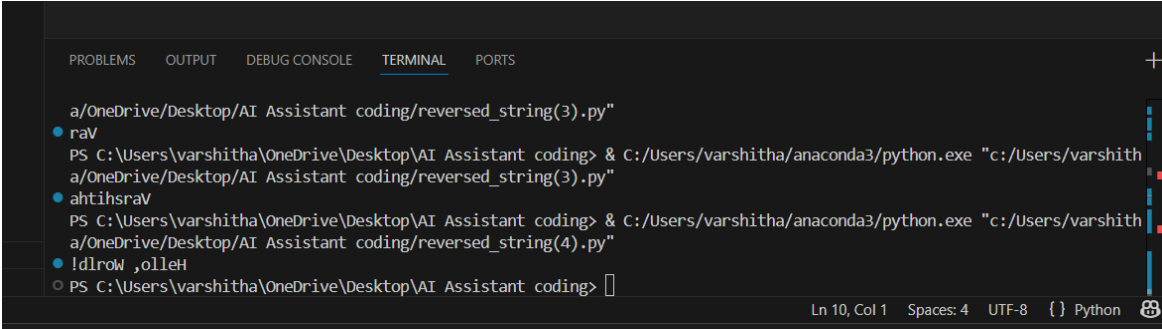
- The user inputs a string.
- An empty string rev is created.
- The loop reads each character from left to right.
- Each character is added at the beginning of rev, reversing the order.

- The reversed string is printed.
- This method helps understand string manipulation logic.

#Generate a slicing based string reversal program in Python



Output:



Explanation

- The string is taken from the user.
- Python slicing reverses the string efficiently.
- The reversed string is printed directly.
- This approach is best for large inputs and real-world applications.

Comparison of Approaches

Aspect	Loop-Based	Slicing-Based
Execution Flow	Step-by-step reversal	Single operation
Time Complexity	O(n)	O(n)
Performance for Large Inputs	Slower	Faster
Readability	Moderate	Very High
Best Usage	Learning logic	Production code